

HelixConstitution

HelixConstitution

La constitution universelle d'ingénierie que tout projet hérite — loi anti-bluffer, appliquée mécaniquement, partagée sous forme de sous-module Git.

Résumé

HelixConstitution est le recueil de règles unique et indépendant des projets — ajouté en tant que sous-module Git par chaque projet Helix/vasic-digital — qui encode une discipline d'ingénierie non négociable (anti-bluffer, validation par preuves uniquement, sécurité des données/hôtes, documentation et couverture de tests) et la propage à une flotte de plus de 140 dépôts. Il constitue l'épine dorsale de gouvernance qui confère une cohérence à l'ensemble de la famille.

Description courte

Un Constitution universel et héritable, distribué sous forme de sous-module Git. Il définit des règles obligatoires et non négociables — barrières anti-bluffer fondées sur des preuves, immunité aux faux positifs, sécurité des données et des hôtes, discipline en matière de couverture et de documentation — que tout projet consommateur hérite automatiquement et peut étendre, mais jamais affaiblir.

Description longue

HelixConstitution est la source canonique et unique de vérité pour les pratiques d'ingénierie partagées par tous les projets qui choisissent de l'adopter en l'ajoutant comme sous-module Git — une loi d'ingénierie, distribuée et versionnée exactement comme du code. Son élément central — `Constitution.md` — est un document d'environ 1 Mo, versionné en continu, composé de clauses numérotées (la famille de conventions §11.4.x,

actuellement jusqu'à §11.4.170) ainsi que de manuels opérationnels par agent (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md) qui l'importent par référence, de sorte que les humains et chaque agent CLI s'appuient sur un seul et même recueil de règles. L'héritage est délibérément structuré en trois couches : la base universelle (ce sous-module), la couche projet (un Constitution/CLAUDE/AGENTS propre au projet qui l'étend) et une couche facultative par sous-répertoire — évaluées de haut en bas, où un projet peut *renforcer* les règles, mais est architecturalement empêché de les *affaiblir*. Le résultat est une flotte de plus de 140 dépôts qui ne peuvent dériver silencieusement, car la discipline qu'ils partagent est figée, et non simplement mémorisée.

Le document est intransigeamment agnostique quant au domaine : tout ce qui nomme un fournisseur spécifique, une référence matérielle, un port ou une version de bibliothèque doit être relégué dans le Constitution propre au projet consommateur, et l'universalité n'est jamais présumée — elle doit être *méritée* au regard d'un test explicite en quatre parties avant qu'une règle ne soit autorisée dans la base. Sa colonne vertébrale philosophique est l'anti-bluffer, exprimée sous la forme d'une famille imbriquée de conventions — §1.1 immunité aux faux positifs, §11.4 convention de qualité pour l'utilisateur final, §11.4.6 interdiction des suppositions, §11.4.69 taxonomie des preuves positives — dont l'effet combiné trace une ligne rouge unique : le seuil de livraison n'est jamais « les tests passent », mais « un utilisateur réel peut utiliser la fonctionnalité », et tout résultat positif doit citer une preuve physique capturée, sans quoi il ne compte pas. Un document compagnon, submodules-catalogue.md (142 dépôts), transforme la question « disposons-nous déjà d'un outil qui fait cela ? » en réflexe de consultation du catalogue avant d'écrire une seule ligne de nouveau code. Des scripts auxiliaires localisent le sous-module à n'importe quelle profondeur d'imbrication et diffusent chaque commit vers quatre fournisseurs Git indépendants, de sorte que le recueil de règles unique et faisant autorité soit également impossible à perdre.

Contenu

Pourquoi nous l'avons conçu

Plusieurs grandes applications produits et des dizaines de sous-modules réutilisables découplés, tous signés par le même propriétaire, ne cessaient de réinventer les mêmes règles durement acquises — et de reproduire la même catégorie d'échecs : des tests et des rapports de statut affichant un succès alors que la fonctionnalité était défailante pour l'utilisateur final (« faux positifs » et « faux négatifs masqués »). Chaque ancrage médico-légal dans le Constitution documente un incident réel (comme le faux positif du routage audio D3 du 20 mai 2026, où la validation est passée au vert avec un champ « Codec en cours

d'utilisation » vide, ou l'interface à bouton géant du 25 juin 2026, qui a réussi les tests d'égalité des jetons alors que l'écran réel était corrompu). Le Constitution a été créé pour rendre cette classe entière de succès mensongers mécaniquement impossible, une fois pour toutes et de manière universelle — afin que cette discipline ne puisse plus dériver entre les projets ni être discrètement oubliée.

Pourquoi c'est un changement radical

Il transforme la culture d'ingénierie, passant de documents que les équipes espèrent suivre à un corpus de lois héritées, versionnées et appliquées mécaniquement — la différence entre un guide de style et un compilateur. Une simple mise à jour de sous-module actualise les règles pour l'ensemble du parc en une seule fois, de manière atomique et traçable. Un seul pacte anti-faux-positif est *garanti* présent dans chaque dépôt consommateur, non par confiance, mais par construction : une porte de propagation vérifie littéralement la présence du numéro de clause dans tout le parc, et un test de mutation associé prouve que la porte elle-même n'est pas un leurre — si bien que même l'application des règles est elle-même appliquée. La gouvernance cesse d'être une aspiration consignée sur un wiki que personne ne lit pour devenir un fait auditable et testable, sur lequel on peut pointer un job CI.

Ce qui est innovant

- **Le Constitution en tant que sous-module** — la loi d'ingénierie est distribuée et versionnée exactement comme du code, avec des tags délibérés de type v1.0.0 et un épinglage par projet, de sorte que chaque dépôt sait *précisément* à quelle révision de la loi il est lié.
- **L'anti-faux-positif comme doctrine médico-légale de premier plan** — chaque clause remonte à un mandat opérationnel mot pour mot et, souvent, à l'incident réel qui l'a motivée, si bien que le recueil de règles se lit comme une jurisprudence plutôt que comme une opinion.
- **Le méta-test des règles elles-mêmes (§1.1)** — chaque porte est couplée à une mutation qui doit faire basculer le statut de PASS à FAIL, de sorte que « la porte n'est pas un simulacre » n'est pas une assertion, mais une preuve apportée à chaque exécution ; une porte qui ne peut jamais échouer est considérée comme pire qu'aucune porte du tout.
- **Une universalité méritée** — un test explicite en quatre parties détermine si une règle est véritablement universelle ou simplement spécifique à un projet, maintenant ainsi la base légère, portable et exempte de fuites liées à des fournisseurs.

Comment il est utilisé dans tous les produits (les pouvoirs qu'il confère)

En tant que **pilier obligatoire de gouvernance**, le HelixConstitution n'est pas un document que la famille consulte — c'est la structure porteuse sur laquelle la famille est construite :

- **Épine dorsale de la gouvernance** : chaque projet Helix/vasic-digital l'intègre comme sous-module et l'importe depuis CLAUDE.md / AGENTS.md / QWEN.md / son propre Constitution.md ; les règles s'appliquent sans condition, dès le premier commit, sans possibilité de dérogation par projet.
- **Portes et mandats** : il définit le modèle de couverture à quatre couches — présence dans le code source, survie à la compilation, comportement à l'exécution, porte non truquée — qu'une fonctionnalité doit valider sur les quatre niveaux pour être considérée comme terminée, ainsi qu'une liste croissante de mandats nommés : gestion des identifiants (§11.4.10), synchronisation permanente de la documentation (§11.4.60), mandat des sous-modules conteneurs (§11.4.76), CodeGraph (§11.4.78), couverture obligatoire des types de tests (§11.4.169), et bien d'autres.
- **Propagation** : les portes CM-COVENANT-114-NNN-PROPAGATION vérifient que le *texte littéral* de la clause est présent dans tout le parc consommateur, empêchant ainsi qu'un pacte soit discrètement abandonné dans un recoin du système ; la non-conformité bloque irrévocablement une release, sans échappatoire possible.
- **Découverte** : submodules-catalogue.md transforme la question « disposons-nous déjà d'un module qui fait X ? » en une réponse immédiate, évitant ainsi de dupliquer les efforts dès la phase de conception.
- **Cohérence des agents AI** : la même loi est exprimée de manière identique pour chaque agent CLI (Claude Code, Codex/Cursor/Aider/OpenCode/Crush/Kimi via AGENTS.md, Qwen Code via QWEN.md), si bien que, quel que soit l'outil qui manipule le code, il obéit à un seul et même pacte.

Contenu

Principaux défis techniques et leurs solutions

- **Localisation du sous-module à une profondeur d'imbrication arbitraire** — une règle enfouie trois sous-modules plus bas doit toujours pouvoir trouver la loi sans savoir où elle réside → `find_constitution.sh` remonte les répertoires parents et suit récursivement le pointeur du superprojet Git, en respectant une redéfinition `CONSTITUTION_DIR`

et deux structures prises en charge (constitution/, submodules/ constitution/), garantissant une résolution déterministe quelle que soit la profondeur d'imbrication.

- **Maintenir un dépôt unique comme référence sur quatre fournisseurs Git** — des miroirs inutiles s'ils divergent → `install_upstreams.sh` lit les dépôts distants déclaratifs `Upstreams/*.sh` et configure origin avec plusieurs URL de push, de sorte qu'un simple `git push` se répercute de manière atomique sur GitHub (principal), GitLab, GitFlic et GitVerse, sans qu'aucun miroir ne puisse prendre de retard.
- **Éviter l'inflation des règles et les fuites de projet dans la base universelle** — chaque tentation d'« ajouter ça ici » érode la portabilité → le test en quatre parties de l'universalité acquise, ainsi que la classification universelle vs. projet (§11.4.17), est appliqué à *chaque* nouvelle règle, renvoyant les préoccupations spécifiques au projet vers la couche projet où elles doivent figurer.
- **Prouver que la barrière d'héritage fonctionne réellement** — une barrière que l'on ne voit jamais échouer est une barrière en laquelle on ne peut avoir confiance → `meta_test_inheritance.sh`, un méta-test sentinelle, supprime délibérément l'ancre §11.4 et vérifie que la barrière la détecte, assurant ainsi que le mécanisme de contrôle est continuellement revalidé contre les ruptures silencieuses.

Pile technologique

- **Héritage via Git-submodule** — *pourquoi* : les sous-modules Git sont le seul mécanisme permettant à un recueil de règles d'être à la fois faisant autorité *et* versionné par consommateur, mis à jour via une incrémentation explicite et vérifiable plutôt que par un copier-coller silencieux ; *comment* : les projets consommateurs ajoutent le sous-module et `@import` ses fichiers d'agent, et les trois couches sont évaluées de haut en bas avec un contrat strict d'extension (et non d'affaiblissement) à chaque frontière.
- **`find_constitution.sh`** — *pourquoi* : les règles sont inutiles si le code profondément imbriqué ne peut les localiser de manière fiable, et le codage en dur des chemins casserait dès qu'un projet serait réorganisé ; *comment* : une remontée des répertoires parents couplée à une récursion `git rev-parse --show-superproject-working-tree`, sécurisée par une redéfinition `CONSTITUTION_DIR`, résolvant les deux structures prises en charge.
- **`install_upstreams.sh` + `Upstreams/`** — *pourquoi* : une redondance sur quatre fournisseurs n'est réelle que si elle ne demande aucun effort supplémentaire, sinon les miroirs se dégradent ; *comment* : des fichiers `.sh` déclaratifs par dépôt distant sont matérialisés en un seul origin multi-URL, réduisant quatre pushes en un seul.

- **Méta-tests de mutation §1.1** — *pourquoi* : une barrière qui ne peut jamais échouer est pire qu’aucune, car elle génère une fausse confiance ; *comment* : chaque barrière est associée à une mutation (suppression/renommage via sed) qui doit faire passer le statut de PASS à FAIL, puis est restaurée, prouvant ainsi que chaque barrière mord encore à chaque exécution.
- **Barrières de propagation (CM-COVENANT-114-NNN-PROPAGATION)** — *pourquoi* : un pacte n’est universel que s’il est vérifiable dans *chaque* consommateur, et pas seulement dans le dépôt phare ; *comment* : une recherche littérale du numéro de clause dans les consommateurs, soutenue par un méta-test §1.1 couplé prouvant que le contrôle de propagation peut lui-même échouer.
- **submodules-catalogue.md (§11.4.74)** — *pourquoi* : le moyen le plus rapide de violer la discipline anti-duplication est d’ignorer ce que l’on possède déjà ; *comment* : un inventaire de 142 dépôts, regroupés par capacité, avec une vérification du catalogue enregistrée dans le tracker *avant* tout nouveau scaffolding.
- **Export multi-format** — *pourquoi* : une même loi doit être aussi bien consultable par des humains qui la lisent, des outils qui la parsent, que par des archives qui la préservent ; *comment* : chaque document canonique est généré aux formats .md / .html / .pdf / .docx à partir d’une seule source.

Contenu

Notes sur le statut et la transparence

- **Statut : déployé.** Versionné activement et utilisé comme sous-module dans l’ensemble de la flotte (dépôts publics canoniques et miroirs).
- **Licence : à déterminer** — non explicitement mentionnée dans le matériel source examiné ; à confirmer avec le fichier LICENSE du dépôt avant publication.
- Miroirs en amont supplémentaires : GitLab helixdevelopment1/helixconstitution, GitFlic helixdevelopment/helixconstitution, GitVerse helixdevelopment/HelixConstitution.

Niveau de priorité : Helix-primaire — un pilier de gouvernance obligatoire pour la construction de tout ce qui relève de la famille Helix.